How the Spread Is Computed

A step-by-step walkthrough of the
Cartea–Jaimungal market maker

from L2 order-book tick data to a resting limit order

$$\delta^{\pm,*}(t,q) \;=\; \frac{1}{\kappa^{\pm}} \;+\; \bar{\varepsilon}^{\pm} \;-\; h(t, q \mp 1) \;+\; h(t,q)$$

rent + adverse selection + inventory pressure

Cartea, Jaimungal & Penalva (2015),
Algorithmic and High-Frequency Trading, §10.4.1

as implemented by the standalone Rust runtime
and its Python replay/reference path

Every code listing in this document is checked against the repository line by line, and every number in the worked example is recomputed by running the pricing code, by `scripts/verify_spread_doc.py`.

Contents

1 Orientation: what we are computing, and why

1.1 The job

There are two ways to trade on an exchange, and the whole subject rests on the difference.

Limit order

> "I will buy at 100, and I am willing to wait." It joins a queue — the **order book** — and sits there until somebody chooses to trade against it, or until you cancel. You name your price but not your timing. Placing one *provides* liquidity, which is why the trader who does it is called the **maker**.

Market order

> "Buy me 10 units, now, at whatever the book is showing." It executes immediately against the resting limit orders. You name your timing but not your price. This *consumes* liquidity, so this trader is the **taker**.

Exchanges normally charge the taker more than the maker, and sometimes pay the maker, precisely because resting orders are what makes a market usable.

A *market maker* posts limit orders on both sides at once and waits. It offers to buy at the **bid** and to sell at the **ask**. If somebody's market order sells to us at the bid and somebody else's buys from us at the ask, we have bought low and sold high without ever predicting the price. The gap we collect is the **spread**. From here on, **MO** is short for market order.

Every exchange publishes a *best bid* and a *best ask* — the highest price anyone is currently willing to buy at and the lowest anyone is willing to sell at. Halfway between those two **market** prices sits the **mid-price** S. Note whose prices those are: S is a property of the market, not of us. Our own quotes normally rest outside the best bid and ask and do not move S. It is convenient to describe each quote by how far it sits *from* the mid, rather than by its absolute price. That distance is the **half-spread**, or **depth**, written δ:

$$\text{bid price} = S - \delta^-, \qquad \text{ask price} = S + \delta^+.$$

Everything in this document is about choosing those two numbers, δ^+ and δ^-, at every instant. That single choice is the strategy.

Intuition

Quoting is a trade-off with exactly two sides. Quote *close* to the mid and you get filled often but earn very little per fill. Quote *far* from the mid and each fill is lucrative but they rarely happen. There is an optimum, and the model in this document computes it.

1.2 The two ways this loses money

If earning the spread were the whole story, the answer would be "quote as tight as possible and collect volume". It is not, because of two costs.

1. Inventory risk. Fills do not arrive in neat pairs. Sell a few times in a row without buying and you are **short** — your position is negative, meaning you have sold units you did not hold and will have to buy them back. **Long** is the opposite. Either way the position is exposed to the price moving against you, and that exposure has nothing to do with market making – it is a directional bet you did not choose to take. We write the position, in units, as q; it swings through zero in both directions all day, so every formula below has to work for negative q as well as positive. Large $|q|$ is dangerous.

2. Adverse selection. This one is subtler and it is the reason this model exists. Your quotes get filled *by somebody*, and that somebody chose to trade now, at your price. On average they know something. When a large buy order arrives it eats the cheapest resting sell orders first, then the next cheapest, and so on — it *walks up* the book, or *sweeps* it. On the way it executes (*lifts*) your ask *and* pushes the mid-price up. You sold just before the price rose. The loss is systematic, not bad luck, and it must be priced in.

1.3 The concrete setting

Everything in this document is calibrated on **CASHCAT**, a low-priced token traded on **Hyperliquid**, a crypto derivatives exchange. The instrument is a *perpetual future*: a contract that tracks the token's price, can be held long or short, can be traded with leverage, and — unlike a normal future — **never expires**. That last property matters more than it sounds, because the model in §2 has a deadline T and our instrument does not. §5 has to decide what to do about that.

One fact about the implementation is needed early. The current trader is the standalone Rust runtime in `rust_live/`: one process holds a signed inventory and computes both sides from one market snapshot. The Python listings below are the executable replay/reference path because they are compact enough to explain line by line. They are not the live executable; Rust implements the same model independently, and `rust_live/tests/python_parity.rs` checks selected calibration, HJB, and rounded-quote outputs against the Python path.

1.4 The answer, in one line

The model's optimal ask depth is

$$\delta^{+,*}(t,q) \;=\; \underbrace{\frac{1}{\kappa^{+}}}_{\substack{\text{how far you can}\\ \text{push before}\\ \text{fills dry up}}} \;+\; \underbrace{\bar{\varepsilon}^{+}}_{\substack{\text{expected adverse}\\ \text{selection cost}}} \;+\; \underbrace{\big(h(t,q)-h(t,q-1)\big)}_{\substack{\text{what you charge for}\\ \text{giving up one unit}}}$$

and the bid is its mirror image. The star means *optimal*: δ^{+} is any depth we might choose, $\delta^{+,*}$ is the best one. The first two terms are constants read straight off the market. The third is the interesting one. A **value function** is the best total profit you can still expect to earn from here to the end, given where you are now; $h(t,q)$ is that number for a maker who is at time t holding q units and who will quote optimally from now on. So the third term is the difference in h between holding q units and holding $q-1$ — what you demand in exchange for parting with a unit — and it is what makes the quotes lean. Long too much inventory? Then being one unit lighter is worth more than being where you are, the third term goes negative, and the ask tightens until somebody takes it.

1.5 The pipeline

Intuition

Stage 2 is expensive and runs about every 30 seconds. Stage 3 is a table lookup and runs on every cycle. That split is the whole architecture: solve the hard problem occasionally, read the answer constantly.

1.6 How to read this document

Every implementation section below follows the same shape:

Stage	What happens	Output	Where
1	Fit the market's parameters from L2 tick data	$\kappa^{\pm}, \lambda^{\pm}, \bar{\varepsilon}^{\pm}, \sigma^2$	§3
2	Solve the control problem	$h(t,q)$ and $\delta^{\pm,*}(t,q)$	§4
3	Read the answer at *this* time and *this* inventory	one δ per side	§5
4	Turn the depth into a price the exchange will accept	a bid and an ask	§6
5	Post it, subject to safety checks	a resting order	§6

Table 1: The five stages. Stages 1 and 2 run on a background cadence; stages 3–5 run every quoting cycle.

This is what the book says (equation, with its number)

↓

This is the executable Python reference (quoted verbatim)

↓

This is where the two differ, if they do

Nothing is paraphrased into pseudo-code. Every listing is the real file, checked line by line by `scripts/verify_spread_doc.py`. Where the implementation and the book genuinely part company it is flagged in an amber box, and §8 collects every such case in one list. The complete correspondence is:

The book	What it says	Python reference	§
eq. (10.22)	mid jumps on each MO	`get_epsilon.py` fits $\bar{\varepsilon}^{\pm}$	3
$\lambda e^{-\kappa\delta}$	fills decay in depth	`estimator_common.fit_kappa_survival`	3
eq. (10.2)	inventory jumps by 1	`mm_core.inventory_to_q`	5
eq. (10.26)	the equation for h	`hjb.compute_h_asymmetric`	4
eq. (10.27)	the optimal depths	`hjb._depths_from_h`	4
eq. (10.28)	the matrix $\mathbf{A}$	`hjb.compute_h_symmetric`	4
eq. (10.29)	$\omega = e^{\mathbf{A}(T-t)}z$	eigendecomposition in `hjb.py`	4
$\phi = \frac{1}{2}\sigma^2\gamma$	risk aversion ≡ penalty	`mm_core.effective_phi`	4.1
$\delta^*(t,q)$	the control	`mm_core.select_delta`	5
—	*(nothing — ours)*	`mm_core.assemble_half_spread`	6

Table 2: Book to code. The last row has no book equation behind it: fees, cushions and clamps are entirely our addition, and §8 says so plainly.

2 The model

This section follows Cartea, Jaimungal and Penalva (2015), *Algorithmic and High-Frequency Trading*, Chapter 10. The market-making problem with adverse selection is §10.4.1, "Impact of Market Orders on Midprice", printed pages 262–264. Equations are numbered as in the book. The prose is our own.

2.1 Notation

Table 3 is worth a bookmark; the rest of the document assumes it.

Symbol	Meaning	In code	Units
S_t	mid-price	`mid`	USDC
$\delta^{\pm}$	depth of our ask (+) / bid (−)	`delta_plus/minus`	USDC
q	inventory, in units	`q`	—
Q_t	inventory process	—	—
X_t	cash	—	USDC
$\lambda^{\pm}$	market-order arrival rate	`lambda+/-`	1/s
$\kappa^{\pm}$	fill-intensity decay	`kappa+/-`	1/USDC
$\bar{\varepsilon}^{\pm}$	mean mid-price jump on an MO	`epsilon+/-`	USDC
σ^2	mid-price variance rate	`sigma2_per_sec`	USDC^2/s
ϕ	running inventory penalty	`phi_effective`	USDC/s
α	terminal inventory penalty	`alpha_effective`	USDC
T	horizon	`hjb_horizon_seconds`	s
τ	time-to-go, $\tau = T - t$	`tau_remaining`	s
$\underline{q}, \bar{q}$	inventory bounds	`q_min`, `q_max`	—
$h(t,q)$	reduced value function	`h`	USDC
$\omega(t,q)$	linearised value function	—	—

Table 3: The superscript $\pm$ always refers to the *market order* that fills us: a buy MO (+) lifts our ask, a sell MO (−) hits our bid.

Easy to get backwards

δ^+ prices the **ask** and δ^- prices the **bid**. The sign refers to the incoming market order, *not* to the direction our inventory moves. In code, `select_delta(hjb, q, "bid")` reads `delta_minus` – which looks inverted until you remember that a bid fill *raises* inventory, so the bid is priced off the $q \to q+1$ arm of the equations.

2.2 How fills arrive

Market orders arrive as a *Poisson process* with rate $\lambda^{\pm}$ — meaning they turn up at random, independently, at an average of $\lambda^{\pm}$ per second, with no memory of when the last one came. Whether one reaches *our* order depends on how deep we posted. The model assumes the probability decays exponentially in depth, so the rate at which our quote at depth δ gets filled is

$$\text{fill rate}(\delta) \;=\; \lambda^{\pm}\, e^{-\kappa^{\pm}\delta}. \tag{D1}$$

Intuition

$1/\kappa$ is the natural unit of depth. Quote $1/\kappa$ further out and your fill rate falls by a factor of $e \approx 2.72$. In the pinned 2026-08-17 CASHCAT snapshot $\kappa^+ \approx 10{,}123$ USDC^{-1}, so $1/\kappa^+ \approx 9.9 \times 10^{-5}$ USDC. That is 9.3 **bps** of a 0.1057 mid — a *basis point* is one hundredth of one per cent, i.e. 10^{-4} of the price, and depths are quoted in bps throughout this document because it makes them comparable across instruments at wildly different price levels. The code stores them in USDC. Push the ask 9.3 bps out and you get filled about a third as often.

2.3 How the mid-price moves: adverse selection

This is what distinguishes §10.4.1 from the simpler market-making models earlier in the chapter. The mid-price is not a pure random walk; it *jumps* when a market order arrives, in the direction of that order:

$$dS_t \;=\; \sigma\, dW_t \;+\; \varepsilon^+ dM_t^+ \;-\; \varepsilon^- dM_t^-, \tag{10.22}$$

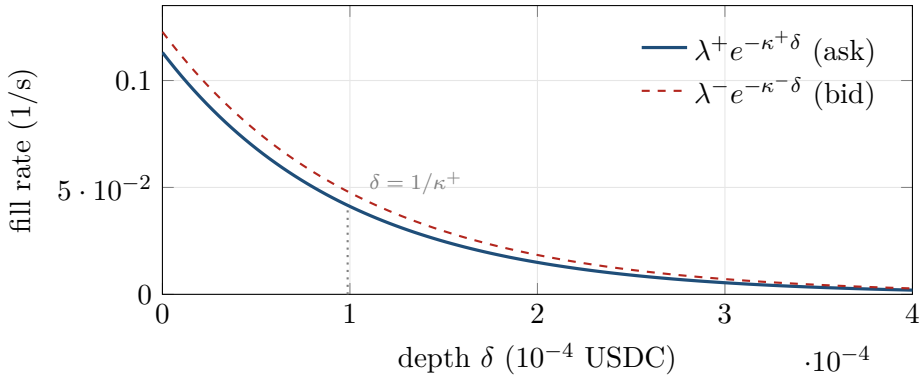

Figure 1: Fill rate against depth at the pinned 2026-08-17 CASHCAT calibration. At $\delta = 1/\kappa$ the rate has fallen to λ/e.

Read that as a recipe for one instant. Over a tiny interval the mid moves for two separate reasons. The term $\sigma\, dW_t$ is ordinary random drift: W is a *Brownian motion*, the continuous-time random walk, and over an interval dt this contributes a mean-zero random amount with variance $\sigma^2 dt$. That is the only fact about W this document uses. The term $dM_t^{\pm}$ is 1 if a buy/sell market order arrived in that instant and 0 otherwise, so $\varepsilon^+ dM_t^+$ adds a jump of size ε^+ exactly when a buy order arrives. Add the two and you have the whole price model.

$M^{\pm}$ count buy and sell market orders, and the jump sizes $\varepsilon^{\pm}$ are i.i.d. draws with means $\bar{\varepsilon}^{\pm} = \mathbb{E}[\varepsilon^{\pm}]$. The distribution is deliberately left unspecified: nothing downstream uses anything about $\varepsilon^{\pm}$ except its mean, which is why §3 only ever has to estimate an average. Throughout, an unbarred ε is the random jump and a barred $\bar{\varepsilon}$ is its mean.

Intuition

Read (D1) and (10.22) together and the problem is obvious. A buy market order is exactly the event that (a) fills our ask and (b) pushes the mid up by ε^+. We sell, and then the price rises. Being filled is itself bad news. The model's answer is to quote $\bar{\varepsilon}^+$ further out so the average fill is compensated for the average subsequent move.

2.4 Inventory and cash

Each fill moves inventory by exactly one unit. The book's (10.2) states this as $Q_t = N_t^- - N_t^+$; in differential form, which is how it is usually quoted,

$$dQ_t \;=\; dN_t^- - dN_t^+, \qquad \text{(cf. 10.2)}$$

where N^- counts our filled bids and N^+ our filled asks. Note this is a *unit-jump* process: q is an integer, and the whole model is defined only on integers. That assumption returns to bite in §5.

Cash accrues at the quoted price on each fill, and the position is bounded to $\underline{q} \le q \le \bar{q}$ – at the boundary the model simply stops quoting the side that would push it further out.

2.5 What the agent is trying to maximise

The performance criterion (p. 262) is

$$H^{\delta}(t,x,S,q) \;=\; \mathbb{E}_{t,x,S,q}\left[X_T^{\delta} + Q_T^{\delta}(S_T - \alpha Q_T^{\delta}) - \phi \int_t^T (Q_u^{\delta})^2 du \right], \qquad \text{(D2)}$$

and the value function is $H = \sup_{\delta^{\pm}} H^{\delta}$.

That supremum is not over two numbers. It is over entire *strategies*: a choice of δ^+ and δ^- for every future instant and every inventory we might find ourselves holding. That is what makes the problem hard, and it is why the answer comes out as a *function* $\delta^*(t,q)$ rather than a pair of constants. (The subscripts on $\mathbb{E}$ mean "given that right now the time is t, cash is x, the mid is S and inventory is q".)

Three terms:

X_T^δ the cash collected from making markets. The thing we want.

$Q_T(S_T - \alpha Q_T)$
liquidate whatever is left at T at the mid, paying a penalty αQ_T^2 for the privilege. α prices the cost of dumping a position at market.

$-\phi \int (Q_u)^2 du$
a *running* charge for holding inventory at all. Every second spent with $q \neq 0$ costs ϕq^2.

Intuition

ϕ and α are the two risk dials. ϕ says "I dislike holding inventory *at any moment*"; α says "I dislike being caught holding inventory *at the end*". Their *ratio* decides whether the strategy unwinds steadily throughout, or procrastinates and flattens near the deadline. We return to this in §5.4, where it turns out our calibration is overwhelmingly ϕ-dominated.

2.5.1 Where ϕ comes from: the utility-maximising maker

A reasonable objection to (D2) is that $\phi \int q^2$ looks arbitrary. Why penalise inventory quadratically, in a criterion that is otherwise about expected cash?

The book answers this in §10.3, "Utility Maximising Market Maker". Suppose instead the agent has no running penalty at all, and simply maximises the expected *exponential utility* $u(x) = -e^{-\gamma x}$ of terminal wealth, with risk aversion γ. Working through the corresponding equation gives optimal depths

$$\delta^{*,\pm} = \frac{1}{\gamma}\log\left(1 + \frac{\gamma}{\kappa^\pm}\right) + g(t,q) - g(t, q \mp 1), \tag{10.19}$$

which is the same shape as the running-penalty answer, with $1/\kappa^\pm$ replaced by $\hat{\kappa}^{\pm -1} = \gamma^{-1}\log(1+\gamma/\kappa^\pm)$ — a risk-aversion bias that vanishes as $\gamma \downarrow 0$. Matching the two problems term by term, the book finds they are the *same problem* under

$$\boxed{\phi = \tfrac{1}{2}\sigma^2\gamma, \qquad \lambda_0 = e^{+1}\hat{\lambda}, \qquad \kappa_0 = \kappa} \tag{D3}$$

(an unnumbered display on p. 261; the consequences are its eqs. 10.20 and 10.21). With that re-scaling the optimal strategies differ only by the constant $(\hat{\kappa}^\pm)^{-1} - (\kappa_0^\pm)^{-1}$, and the value functions are related by $H = -\gamma^{-1}\log(-G)$.

Intuition

This is worth pausing on. The running penalty is not a hack – it is *equivalent* to honest risk aversion, and the exchange rate between them is $\phi = \frac{1}{2}\sigma^2\gamma$. In particular ϕ should be **proportional to variance**: the same risk aversion in a more volatile market implies a larger inventory penalty. That is exactly what the code's volatility channel does (§4.1), and (D3) is its justification.

Departure from the book

Two honest caveats on borrowing this result. First, the book proves it for the *base* model of §10.2, which has no adverse-selection jumps — there σ is the whole of the price movement, whereas in §10.4.1 part of the variance is the $\varepsilon^{\pm}$ jumps. Second, and for the same reason, the σ^2 we measure is the *total* mid variance including those jumps, so the volatility channel mildly double-counts adverse selection. At the pinned calibration that channel is about 6% of ϕ, so the effect is small — but it is there, and (D3) is a motivation for the term rather than a derivation of it.

2.6 Reducing the problem

The value function H depends on four variables, which is two too many. The book's ansatz strips out cash and price:

$$H(t, x, S, q) \;=\; x + qS + h(t, q). \tag{10.25}$$

Cash is worth its face value, the inventory is worth the mid, and everything interesting is in $h(t, q)$, a function of *time and an integer*.

Intuition

It is worth being concrete about h, because every interesting number in the rest of this document is a difference of it, and it is the only object here with no direct market counterpart.

Ansatz (10.25) says your worth is cash, plus inventory marked at the mid, plus $h(t, q)$. So $h(t, q)$ is **everything not already on your balance sheet**: in USDC, how much better or worse off you expect to end up than a trader who is handed your exact position, marks it at the mid, and stops trading on the spot. It is the value of still being in business, given that you are in business holding q.

Two consequences get used constantly below. First, h is largest near flat inventory and falls away on both sides — holding a big position is a worse place to be trading from. Second, the difference $h(t, q) - h(t, q-1)$ is what you should charge to go from q units to $q-1$: not because a unit is worth that, but because your whole position is worth that much more once it is one unit lighter. That difference is the third term of the answer, and §7 shows it dwarfing the other two.

Substitute this into the **Hamilton–Jacobi–Bellman (HJB)** equation — the book's (10.23), and the reason the solver file is called `hjb.py`. The HJB equation is the continuous-time statement of dynamic programming: over the next instant the value function must change at exactly the rate justified by the best action available now, so that no better policy exists. We do not reproduce (10.23) itself because nothing downstream reads it, but the mechanical content of the substitution is short: differentiate $H = x + qS + h(t, q)$, note that $\partial_x H = 1$ and $\partial_S H = q$ so the cash and price terms are linear and cancel against the drift, and what survives is an equation in t and q alone — four variables collapsed to two:

$$\begin{aligned} \phi q^2 \;=\; \partial_t h &+ \lambda^+ \sup_{\delta^+} \Big\{ e^{-\kappa^+ \delta^+} \big(\delta^+ - \bar{\varepsilon}^+ + h_{q-1} - h_q\big) \Big\} \mathbf{1}_{q > \underline{q}} \\ &+ \lambda^- \sup_{\delta^-} \Big\{ e^{-\kappa^- \delta^-} \big(\delta^- - \bar{\varepsilon}^- + h_{q+1} - h_q\big) \Big\} \mathbf{1}_{q < \bar{q}} \\ &+ (\bar{\varepsilon}^+ \lambda^+ - \bar{\varepsilon}^- \lambda^-) q, \end{aligned} \tag{10.26}$$

subject to the terminal condition $h(T, q) = -\alpha q^2$. Here and below h_q is shorthand for $h(t, q)$ at the current time.

Intuition

Read the two sup terms as "rate of fills × profit per fill". The profit per fill on the ask is: the depth you collect (δ^+), minus what adverse selection costs you ($\bar{\varepsilon}^+$), plus the change in your position's option value from going one unit shorter ($h_{q-1} - h_q$). Choosing δ^+ trades the rate against the profit, and sup picks the best point.
The last line is a *drift*: even a maker who never trades is exposed, because market orders move the mid and we hold q of the asset. It is small but it is not zero, and it is easy to drop by accident.

2.7 The optimal depths

Each sup in (10.26) is one-dimensional and has a closed form. Writing $\Delta h = h_{q\mp1} - h_q$ for the relevant side, maximise $f(\delta) = e^{-\kappa\delta}(\delta - \bar{\varepsilon} + \Delta h)$:

$$f'(\delta) = e^{-\kappa\delta}\Big[1 - \kappa\big(\delta - \bar{\varepsilon} + \Delta h\big)\Big] = 0 \implies \delta^* = \frac{1}{\kappa} + \bar{\varepsilon} - \Delta h.$$

That is the whole derivation. Written out for both sides:

$$\boxed{\begin{aligned} \delta^{+,*}(t,q) &= \frac{1}{\kappa^+} + \bar{\varepsilon}^+ - h(t,q-1) + h(t,q), \quad q \neq \underline{q}, \\ \delta^{-,*}(t,q) &= \frac{1}{\kappa^-} + \bar{\varepsilon}^- - h(t,q+1) + h(t,q), \quad q \neq \bar{q}. \end{aligned}} \tag{10.27}$$

Substituting δ^* back gives the value of the sup, which we will need in a moment:

$$\lambda\, e^{-\kappa\delta^*}\big(\delta^* - \bar{\varepsilon} + \Delta h\big) \;=\; \frac{\lambda}{\kappa}\, e^{-\kappa\delta^*}. \tag{D4}$$

Intuition

The three terms of (10.27) are the three ideas of this document:

1. $1/\kappa^\pm$ — **rent**. How much you can charge before fills dry up. Set purely by liquidity.
2. $\bar{\varepsilon}^\pm$ — **adverse-selection recovery**. Step back by the average post-fill move, so the average fill breaks even against it.
3. $h(t,q) - h(t,q \mp 1)$ — **inventory pressure**. The only term that knows what you are holding. This is what skews the quotes.

The exclusions $q \neq \underline{q}$, $q \neq \bar{q}$ mean: at the top of the inventory band there is *no bid at all*, and at the bottom there is no ask.

2.8 Solving for h

Equation (10.26) is still non-linear because of the exponentials, but when $\kappa^+ = \kappa^- = \kappa$ it linearises exactly. Substitute (D4) into (10.26) and set

$$h(t,q) = \frac{1}{\kappa}\log\omega(t,q).$$

Then $e^{-\kappa\delta^*} = e^{-1-\kappa\bar{\varepsilon}}\, e^{\kappa\Delta h} = e^{-1-\kappa\bar{\varepsilon}}\, \omega_{q\mp1}/\omega_q$. Define

$$\tilde{\lambda}^\pm \;=\; \lambda^\pm e^{-1-\kappa\bar{\varepsilon}^\pm},$$

which is not merely a grouping of symbols: $\tilde{\lambda}^\pm$ is the fill rate the strategy actually runs at — the raw arrival rate, discounted once for standing $1/\kappa$ back from the mid, and again for standing $\bar{\varepsilon}^\pm$ further back to cover adverse selection. §3 returns to that second factor under the name *toxicity*.

Because $h = \kappa^{-1}\log\omega$ we also have $\partial_t h = \partial_t\omega_q/(\kappa\omega_q)$. Multiply (10.26) through by $\kappa\omega_q$ and every term is linear — note that ϕq^2 crosses the equals sign on the way, so it appears with a minus inside the bracket:

$$\partial_t\omega_q + \tilde{\lambda}^+\omega_{q-1} + \tilde{\lambda}^-\omega_{q+1} + \Big[\kappa q(\lambda^+\bar{\varepsilon}^+ - \lambda^-\bar{\varepsilon}^-) - \phi\kappa q^2\Big]\omega_q \;=\; 0.$$

Stacking the ω_q into a vector, that is $\partial_t\boldsymbol{\omega} + \mathbf{A}\boldsymbol{\omega} = \mathbf{0}$ with

$$\mathbf{A}_{i,q} = \begin{cases} q\kappa(\bar{\varepsilon}^+\lambda^+ - \bar{\varepsilon}^-\lambda^-) - \phi\kappa q^2, & i = q, \\ \tilde{\lambda}^+, & i = q-1, \\ \tilde{\lambda}^-, & i = q+1, \\ 0, & \text{otherwise.} \end{cases} \tag{10.28}$$

The terminal condition $h(T,q) = -\alpha q^2$ becomes $\omega(T,q) = e^{\kappa h} = e^{-\alpha\kappa q^2}$, and a linear ODE with a constant matrix has one answer:

$$\boldsymbol{\omega}(t) \;=\; e^{\mathbf{A}(T-t)}\mathbf{z}, \qquad z_j = e^{-\alpha\kappa j^2}. \tag{10.29}$$

Easy to get backwards

Look at where κ appears in (10.28) and in $z_j = e^{-\alpha\kappa j^2}$: the model responds to the *products* $\phi\kappa$ and $\alpha\kappa$, never to ϕ or α alone. So ϕ **and** α **are not** κ**-invariant**. A ϕ tuned on one asset means something completely different on another whose κ differs by a factor of ten. This is why the code tunes the dimensionless $\phi\kappa T$ and $\alpha\kappa$ instead, and derives ϕ and α from them (§4.1).

3 Stage 1: estimating the parameters from L2 tick data

The model takes $\kappa^\pm, \lambda^\pm, \bar{\varepsilon}^\pm$ (and, through (D3), σ^2) as given. Real markets do not hand them over, so they are fitted continuously from the raw feed.

3.1 The raw data

A collector (`scripts/hyperliquid_data_collector.py`) subscribes to several Hyperliquid public streams. The two streams used by this calibration are written to Parquet shards as follows:

- **L2 book / BBO**: an *L2* feed reports the order book aggregated by price level — how much is resting at each price — rather than order by order. *BBO* is the "best bid and offer", i.e. just the top level of each side. Each update is written as two rows (one `bid`, one `ask`) with price, size, local receive time and exchange time. This is the tick data everything else is measured against.
- **Trades**: every public print — a record that a trade happened — with side and size.

Both are loaded once per estimation cycle by `estimator_common.load_market_window()`, which does the unglamorous work that decides whether the estimates mean anything:

- **One clock.** Exchange timestamps are used only if *every* stream has $\geq 99\%$ coverage of them; otherwise everything falls back to local receive time. Never a mixture.
- **A mid series** is built from the dense BBO stream, forward-filled, keeping only rows with $0 < \text{bid} < \text{ask}$.
- **Trade de-duplication** on trade id. This is not paranoia: on 2026-08-16 two collectors sharing one output directory wrote every tick twice, silently doubling $\lambda^\pm$.
- **Market-order aggregation.** Prints sharing a timestamp and side collapse into one market order, taking the *deepest* price. This is what makes λ "market orders per second" rather than "prints per second", and it is what makes the depth below a *walk* distance.

For each market order, the mid *strictly before* it is attached. The implementation uses `merge_asof` with exact matches disabled, and the walk depth is measured from that mid:

$$\text{depth}^{+} = (\text{highest buy print}) - \text{pre-mid}, \qquad \text{depth}^{-} = \text{pre-mid} - (\text{lowest sell print}).$$

Intuition

Depth is measured **from the mid**, not from the touch. That is deliberate: it is the same coordinate the runtime and replay quote in, so a fitted κ can be fed straight back into δ without a change of variable.

3.2 $\kappa^{\pm}$: how fast fills die off with depth

Here is the key insight, and it is prettier than it first looks. Our order resting at depth δ is filled by a market order exactly when that order *walks at least as deep as* δ. So

$$\text{fill rate}(\delta) \;=\; \Lambda \cdot \Pr[\text{depth} \geq \delta] \;=\; \Lambda \cdot S(\delta),$$

and the model's assumption (D1) is precisely the claim that the **survival function** $\bar{F}(\delta)$ of market-order walk depths is exponential. (We write $\bar{F}$, not S, because S is the mid-price everywhere else in this document.) Take logs and it is a straight line:

$$\log \bar{F}(\delta) \;=\; c - \kappa\delta.$$

So κ is the slope of a log-survival plot – no binning, no fill simulation, just the empirical distribution of how deep market orders reach.

Two idealisations are buried in "exactly when"

A market order that reaches our price level fills the orders *ahead of us in the queue* first, and may be exhausted before it reaches ours; it may also be smaller than our resting size. The model has no notion of queue position, so neither effect can be corrected inside it. The consequence is worth stating plainly: the walk-depth survival function is an **upper bound** on our fill probability, and the fitted κ is correspondingly optimistic. Everything downstream inherits that optimism.

The book: assumption: fill rate$(\delta) = \lambda^{\pm} e^{-\kappa^{\pm}\delta}$

The book takes $\kappa^{\pm}$ as given and never says where it comes from. Our reading is that the assumption is equivalent to "market-order walk depths are exponentially distributed", which *is* something we can measure directly.

→ what we wrote

A weighted log-linear regression of the empirical survival function $\log S(\delta) = c - \kappa\delta$, weighted by $\sqrt{\text{tail count}}$:

`scripts/estimator_common.py` lines 730–759

```
    sorted_depths = np.sort(depths)

    in_support = sorted_depths <= support_cap
    support_floor = float("nan")
    if support_quantile_lower > 0.0:
        support_floor = float(np.quantile(depths, support_quantile_lower))
        in_support &= sorted_depths >= support_floor
```

```
    grid = np.unique(sorted_depths[in_support])
    if len(grid) < 3:
        return _empty(support_floor, support_cap)

    # Tail counts via searchsorted: count of depths >= delta.
    tail_counts = n_total - np.searchsorted(sorted_depths, grid, side="left")
    survival = tail_counts.astype(float) / float(n_total)
    mask = (survival > 0) & (tail_counts >= 2)
    grid = grid[mask]
    survival = survival[mask]
    tail_counts = tail_counts[mask]
    if len(grid) < 3:
        return _empty(support_floor, support_cap)

    y = np.log(survival)
    weights = np.sqrt(tail_counts.astype(float))
    design = np.column_stack((np.ones_like(grid), -grid))
    try:
        coef, _, _, _ = np.linalg.lstsq(design * weights[:, None], y * weights, rcond=None)
    except np.linalg.LinAlgError:
        return _empty(support_floor, support_cap)
    intercept, kappa = float(coef[0]), float(coef[1])
```

The weights are $\sqrt{\text{tail count}}$, because a survival estimate based on two observations in the tail is far less precise than one based on two hundred. Support is capped at the 99th percentile so a single freak sweep cannot lever the slope.

Easy to get backwards

A **buy** market order lifts the **ask**, so buy-side depths calibrate κ^{+} – the parameter that prices *our ask*. Getting this backwards produces a strategy that skews the wrong way and is very hard to spot.

3.3 $\lambda^{\pm}$: how often market orders arrive

Much simpler: count them and divide by the time actually covered.

scripts/get_kappa.py lines 266–274

```
        data_start_ms = max(window.window_start_ms or 0.0, float(window.mids["ts_ms"].min()))
        wall_seconds = max(((window.window_end_ms or 0.0) - data_start_ms) / 1000.0, 1e-6)
        outage_excluded = float((window.meta or {}).get("outage_seconds", 0.0) or 0.0)
        covered_seconds = max(wall_seconds - outage_excluded, 1e-6)

    lambda_plus_raw = n_buy_mos / covered_seconds if covered_seconds > 0 else float("nan")
    lambda_minus_raw = n_sell_mos / covered_seconds if covered_seconds > 0 else float("nan")
    kappa_plus_raw = buy_fit["kappa"]
    kappa_minus_raw = sell_fit["kappa"]
```

Two corrections, and they are not the same one. The window starts at the *later* of the requested start and the first observed mid, so a cold start with a partially filled window does not deflate λ. Interior *outages* are then subtracted as well: a rate is a count over the time we were watching, and dividing by wall clock understates it by exactly the missing fraction. On a two-hour CASHCAT window containing a 405 s outage that is 5.6%; inside the one-hour gap the acceptance gate now tolerates it would be 50%.

A silent price stream is not an outage

On an illiquid instrument the best bid and offer can sit unchanged for minutes while the collector is perfectly healthy. Over 61.2 h of CASHCAT the price stream showed 91.5 minutes of gaps longer than a minute, but only 43.2 of those minutes had *no* stream receiving anything – the rest was a quiet book with trades still arriving. Charging that

(continued)

time to downtime would inflate λ, which is the opposite of the bug being fixed, so the outage census asks the union of the streams rather than the mids alone.

3.4 $\bar{\varepsilon}^{\pm}$: the adverse-selection cost

$\bar{\varepsilon}^{\pm}$ is the average amount the mid moves right after a market order — a *markout*, i.e. the price change measured a fixed interval after an event. It is measured over a 200 ms window:

scripts/get_epsilon.py lines 189–197

```
    post = pd.merge_asof(
        frame,
        mids[['ts_ms', 'mid']].sort_values('ts_ms').rename(columns={'ts_ms': 'mid_ts_ms', 'mid':
     'post_mid'}),
        left_on='target_ms',
        right_on='mid_ts_ms',
        direction='backward',
        allow_exact_matches=True,
    )
    post = post.dropna(subset=['post_mid'])
```

The result is 3σ-clipped and floored at zero (the model requires $\bar{\varepsilon} \geq 0$). Events too close to the end of the window are dropped, so the horizon never runs off the end of the data and biases the mean downwards.

Intuition

Why 200 ms and not 5 s? Because (10.22) defines ε as the jump *at arrival*. A 5-second markout sweeps up every *other* market order that arrived in between, and those are already accounted for by the drift term $q(\lambda^{+}\bar{\varepsilon}^{+} - \lambda^{-}\bar{\varepsilon}^{-})$ in (10.26). Using a long markout here would count the same effect twice. The 1 s and 5 s markouts are computed anyway, but only as diagnostics.

3.5 σ^2: the variance rate

Variance of one-second mid increments, per second:

scripts/estimator_common.py lines 853–859

```
    sampled = np.where(fresh, mid[np.clip(idx, 0, None)], np.nan)

    diffs = np.diff(sampled)
    diffs = diffs[np.isfinite(diffs)]
    if len(diffs) < min_samples:
        return None
    sigma2 = float(np.var(diffs)) / step
```

The `fresh` mask is the interesting part: grid points whose most recent observation is more than 5 seconds stale are voided rather than forward-filled. Without it, a collector outage would appear as a single enormous price jump and read as a volatility spike – which, through (D3), would widen every quote in response to a *data* problem.

3.6 Gating and transport

A cycle that fails its own quality checks is rejected, so one bad window cannot replace a still-fresh valid model. Calibration validates schema/status, source-data freshness, $\kappa > 0$, $\lambda \geq 0$, $\bar{\varepsilon} \geq 0$, fit quality ($R^2 \geq 0.30$, ≥ 6 fit points, ≥ 50 impact events), and a **toxicity** gate $\max(\kappa^{+}\bar{\varepsilon}^{+}, \kappa^{-}\bar{\varepsilon}^{-}) \leq 1.5$. If no fresh valid model remains, quoting fails closed.

Stage	What	Value
Window	trailing data used per fit	120 minutes
Cadence	re-fit interval	30 s
Model input	direct validated rolling-window estimate	no temporal smoothing
Runtime path	Rust calibration to in-process model bundle	`cj-data` → `cj-core`
Python path	explicit JSON artifact for analysis/replay	schema v4
Max age	calibration and source data must remain fresh	120 s

Table 4: The current runtime cadence and data boundary. Python JSON snapshots are analysis artifacts; the Rust trader calibrates directly from Parquet.

Intuition

The toxicity number $\kappa\bar{\varepsilon}$ is dimensionless: adverse-selection cost measured in units of the model's natural depth $1/\kappa$. Inside the idealised HJB, at flat inventory and before fees or clamps, the optimal depth is $1/\kappa + \bar{\varepsilon}$, so the fitted fill rate there is

$$\lambda\, e^{-\kappa(1/\kappa+\bar{\varepsilon})} = \underbrace{\lambda e^{-1}}_{\text{cost of quoting at the optimum}} \times \underbrace{e^{-\kappa\bar{\varepsilon}}}_{\text{cost of adverse selection}} = \tilde{\lambda},$$

which is exactly the $\tilde{\lambda}^{\pm}$ that appears in (10.28).

This identity is useful for diagnosing the fitted model; it is not a profitability theorem. The configured ceiling $\kappa\bar{\varepsilon} \leq 1.5$ is a fail-closed operating heuristic, not a break-even boundary derived in the book. It contains no maker fee, queue position, latency, inventory tail, model error, or parameter uncertainty. Values below the ceiling can lose money, and values above it are not proved unprofitable merely by crossing the line.

Economic viability is checked separately by `scripts/verify_market_viability.py`, which uses measured reachable volume, conditional markout, and fees, followed by event replay. The distinction is observable: during the liquidation cascade in `docs/TOXIC_FLOW_GUARD.md`, $\kappa\bar{\varepsilon}$ remained about 0.25 while the flow was acutely dangerous. It is a rolling calibration property, not a fast flow alarm.

3.7 What the three parameters say about the instrument

Before any of them reach a solver, κ, λ and $\bar{\varepsilon}$ describe the modelled flow regime. They do *not* answer whether the instrument is worth quoting after fees, queueing, and latency. As qualitative model preferences,

	you want	because
λ	high	many market orders, so many chances to earn the spread
κ	low	a thin book, so you can stand further back and still get filled
$\bar{\varepsilon}$	low	little informed flow, so fills are not systematically bad news

In one line, the fitted model prefers frequent arrivals, a heavy tail of market-order walk depths, and small arrival jumps. Whether that combination pays is an empirical question for the viability curve and replay, not for these three point estimates alone.

Note that κ appears twice with opposite signs of preference: low κ is good for the rent $1/\kappa$, but κ also multiplies $\bar{\varepsilon}$ in the toxicity number. A thin book helps only if the flow trading in it is uninformed.

4 Stage 2: solving for h

4.1 Choosing ϕ and α

The book: ϕ and α are free parameters

The book treats ϕ (running penalty) and α (terminal penalty) as constants the agent chooses, and §10.3 pins one of them down: $\phi = \frac{1}{2}\sigma^2\gamma$ for an agent with risk aversion γ.

→ what we wrote

We do *not* configure ϕ and α directly, because Trap 2.8 shows they are not κ-invariant. We configure the dimensionless $\phi\kappa T$ and $\alpha\kappa$ and derive the rest, then add the volatility term that §10.3 calls for:

`scripts/mm_core.py` lines 457–466

```
if kappa_avg > 0:
    # Derive from the dimensionless targets so the same config expresses the
    # same risk trade-off at any kappa.
    if float(config.hjb_phi_kappa_t) > 0 and horizon > 0:
        phi = float(config.hjb_phi_kappa_t) / (kappa_avg * horizon) + vol_delta
        phi_source = "kappa_relative+sigma2" if vol_delta else "kappa_relative"
        ceiling = float(config.hjb_phi_kappa_t_max)
        if ceiling > 0 and phi * kappa_avg * horizon > ceiling:
            phi = ceiling / (kappa_avg * horizon)
            phi_source += "+capped"
```

So the runtime/reference ϕ and α are

$$\phi = \frac{(\phi\kappa T)_{\text{target}}}{\bar{\kappa}T} + \underbrace{\gamma\,\sigma^2\,u}_{\text{volatility channel}}, \qquad \alpha = \frac{(\alpha\kappa)_{\text{target}}}{\bar{\kappa}},$$

with $\bar{\kappa}$ the mean of $\kappa^{\pm}$, u the inventory unit size in base asset, and $\gamma = 0.05$. The volatility channel is the code's realisation of (D3): risk aversion times variance. If σ^2 is missing the term is simply dropped – a missing variance estimate must not fabricate one.

4.2 Choosing the number: what a sweep of $\phi\kappa T$ actually shows

$\phi\kappa T = 10$ was the Python replay default used by this dated sweep. The current Rust profile uses 200 (ceiling 300), so the table below is historical evidence about shape, not a statement of the current configuration.

Replaying the whole pipeline over one pinned 18.67-hour CASHCAT tape (115,364 book updates, 46,072 trades), varying only $\phi\kappa T$ and deliberately running well past the configured ceiling of 50:

Easy to get backwards

The configuration comment in `mm_core.py` records an earlier sweep on a 9.8-hour tape that found "an inverted U with the optimum at 10–20". On this longer tape **there is no inverted U and no interior optimum**. The loss shrinks monotonically as the inventory penalty rises, right out to $\phi\kappa T = 400$ — eight times the configured ceiling — where it is the smallest of the whole sweep.

The honest reading is uncomfortable and worth stating plainly. Look at the fills column of Table 5: 1959 down to 338. Look at USDC per fill: it sits between −0.06 and −0.11 across a factor of 8000 in the parameter. **Each fill costs roughly the same no matter how the model is tuned; all ϕ does is change how many fills you take.** Total

$\phi\kappa T$	PnL (USDC)	maker fills	USDC per fill	
0.05	−114.61	1928	−0.059	
0.20	−133.22	1959	−0.068	
1.00	−135.94	1867	−0.073	worst
3.00	−119.40	1686	−0.071	
10.00	−95.95	1405	−0.068	**historical default**
20.00	−63.58	1121	−0.057	
50.00	−65.09	784	−0.083	ceiling
100.00	−57.75	614	−0.094	
200.00	−47.14	427	−0.110	
400.00	−24.83	338	−0.073	best tested

Table 5: Running-penalty sweep, one pinned tape, quote attempts held at ≈ 60,800 throughout. The best setting tested is the largest one tested.

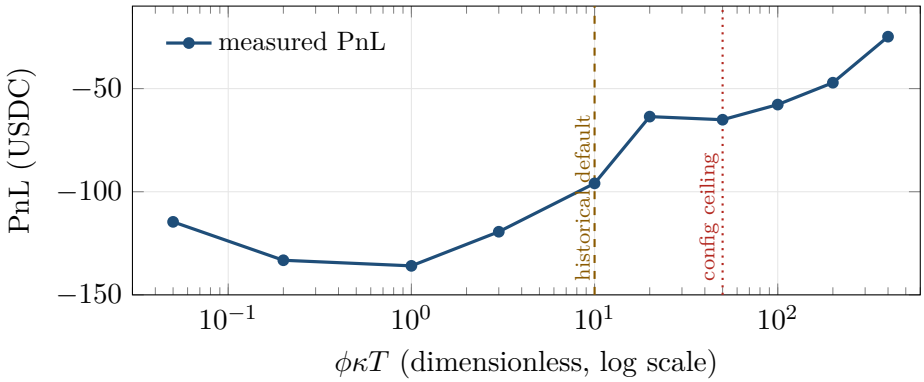

Figure 2: The same data. Past $\phi\kappa T \approx 1$ the loss shrinks steadily — with one reversal, 20 → 50, well inside the run-to-run noise of a single tape — and never turns back up within a factor of 400.

(continued)

loss therefore tracks fill count, not cleverness, and "turning ϕ up" is not finding a better trade-off — it is trading less. The limit of that logic is not quoting at all.
Two consequences. First, do not read $\phi\kappa T = 10$ as a measured optimum; it is a middling point on a monotone curve, and the evidence originally cited for it did not replicate. Second, this sweep is not really measuring the inventory penalty at all — it is evidence that on this instrument and tape, at this simulated latency, the marginal maker fill has negative expected value. No tested setting of ϕ fixes that result.

4.3 Building A

The book: eq. (10.28) and (10.29)

$\mathbf{A}_{i,q}$ has $q\kappa(\bar{\varepsilon}^{+}\lambda^{+} - \bar{\varepsilon}^{-}\lambda^{-}) - \phi\kappa q^2$ on the diagonal, $\tilde{\lambda}^{+}$ and $\tilde{\lambda}^{-}$ off it with $\tilde{\lambda}^{\pm} = \lambda^{\pm}e^{-1-\kappa\bar{\varepsilon}^{\pm}}$; then $\boldsymbol{\omega}(t) = e^{\mathbf{A}(T-t)}\mathbf{z}$ with $z_j = e^{-\alpha\kappa j^2}$.

→ what we wrote

Line for line, with no rearrangement — compare (10.28):

scripts/hjb.py lines 227–241

```
lam_tilde_p = lam_p * np.exp(-1.0 - kappa * eps_p)
lam_tilde_m = lam_m * np.exp(-1.0 - kappa * eps_m)

q_grid = np.arange(q_min, q_max + 1)
d = len(q_grid)
A = np.zeros((d, d))

for i, q in enumerate(q_grid):
    A[i, i] = q * kappa * (lam_p * eps_p - lam_m * eps_m) - phi * kappa * (q ** 2)
    if i > 0:
        A[i, i - 1] = lam_tilde_p
    if i < d - 1:
        A[i, i + 1] = lam_tilde_m

z = np.exp(-alpha * kappa * (q_grid ** 2))
```

Every symbol maps directly: `lam_tilde_p` is $\tilde{\lambda}^{+} = \lambda^{+}e^{-1-\kappa\bar{\varepsilon}^{+}}$, the diagonal is $q\kappa(\lambda^{+}\bar{\varepsilon}^{+} - \lambda^{-}\bar{\varepsilon}^{-}) - \phi\kappa q^2$, and `z` is $z_j = e^{-\alpha\kappa j^2}$.

Easy to get backwards

The off-diagonal placement deserves a second look. `q_grid` is *ascending*, while the book stacks $\boldsymbol{\omega}$ *descending* from $\bar{q}$ down to $\underline{q}$, and the book's case list does not say which index of $\mathbf{A}_{i,q}$ is the row. Do not try to read it off; derive it. The derivation in §2 gives the row at inventory q the term $\tilde{\lambda}^{+}\omega_{q-1}$, because a **buy** MO (rate λ^{+}) fills our ask and takes $q \to q-1$. That is exactly `A[i, i-1] = lam_tilde_p`. The code is right.

4.4 Two solvers

Closed form, when $\kappa^{+} = \kappa^{-}$. Diagonalise $\mathbf{A} = V\Lambda V^{-1}$ and (10.29) evaluates in one shot for every t at once. The implementation carries one wrinkle worth understanding: computing $e^{\mathbf{A}T}$ directly overflows once an eigenvalue times T grows past about 709 in magnitude, returning silent NaNs. Factoring $e^{\theta} = e^{\theta-m}e^{m}$ with $m = \max_j \theta_j$ and adding m back after the logarithm is algebraically exact and cannot overflow.

Backward Euler, when $\kappa^+ \neq \kappa^-$. With different κs the log substitution no longer linearises the problem, so (10.26) is integrated backwards from $h(T, q) = -\alpha q^2$ in `n_steps` implicit steps, each solved by a damped Newton iteration.

Write (10.26) as $\partial_t h = -G(h)$, so G collects the two sup terms, the $-\phi q^2$ and the drift. Because G depends on h only through the neighbour differences $h_{i\pm1} - h_i$, its Jacobian is tridiagonal and known analytically:

$$\frac{\partial G_i}{\partial h_{i-1}} = s^+, \qquad \frac{\partial G_i}{\partial h_i} = -s^+ - s^-, \qquad \frac{\partial G_i}{\partial h_{i+1}} = s^-,$$

where $s^\pm$ is each side's derivative of its sup value with respect to that difference. So each Newton step is a banded solve rather than a dense one. The current Rust runtime and Python reference both use this asymmetric backward-Euler method; the symmetric closed form is an independent limiting-case check in the Python test suite.

Intuition

Why is the derivative free? At the optimum, value $= (\lambda/\kappa)e^{-\kappa\delta^*}$ and $\delta^* = 1/\kappa + \bar{\varepsilon} - \Delta h$, so $\partial(\text{value})/\partial(\Delta h) = \kappa \cdot \text{value}$. The derivative is the function itself times κ – no finite differences anywhere.

4.5 From h to depths

The book: eq. (10.27)

$\delta^{+,*} = \frac{1}{\kappa^+} + \bar{\varepsilon}^+ - h(q-1) + h(q)$ for $q \neq \underline{q}$, and the mirror for the bid at $q \neq \bar{q}$.

→ what we wrote

The subtraction is inside the shared helper `_optimal_delta_and_value`, which returns $\delta^* = 1/\kappa + \bar{\varepsilon} - \Delta h$. What is left here is the loop over q and, more importantly, the two indicator functions:

`scripts/hjb.py` lines 144–158

```
    delta_plus = np.full(d, np.inf, dtype=float)
    delta_minus = np.full(d, np.inf, dtype=float)
    for i in range(d):
        h_q = h_vec[i]
        if i > 0:
            raw_plus, _, _ = _optimal_delta_and_value(
                lam_p, kappa_p, eps_p, h_vec[i - 1] - h_q, clip_at_zero=clip_deltas
            )
            delta_plus[i] = max(0.0, raw_plus) if clip_deltas else raw_plus
        if i < d - 1:
            raw_minus, _, _ = _optimal_delta_and_value(
                lam_m, kappa_m, eps_m, h_vec[i + 1] - h_q, clip_at_zero=clip_deltas
            )
            delta_minus[i] = max(0.0, raw_minus) if clip_deltas else raw_minus
    return delta_plus, delta_minus
```

Note how the book's indicator functions $\mathbf{1}_{q>\underline{q}}$, $\mathbf{1}_{q<\bar{q}}$ are realised: both arrays are *initialised to* $+\infty$, and a side is only written when its neighbour exists. At $q = \underline{q}$ the `if i > 0` guard is false, so the ask stays infinite. Infinity here means **"this side is switched off"**, not "very deep" – a distinction the rest of the pipeline is careful to preserve.

4.6 A normalisation you must know about

Both solvers subtract each time slice's own maximum, so that $\max_q h(t, q) = 0$ on every slice. This is safe because everything downstream reads only *differences within a slice* – (10.27) uses

$h_{q\mp1} - h_q$ and nothing else – and it is *necessary* because in the steady state the unnormalised h drifts linearly in t until its level swamps the $O(1/\kappa)$ differences the depths are made of.

Easy to get backwards

The consequence: in the returned surface, **levels are not comparable across time slices**. Differences within a slice are exact. Do not plot `h_surface` as if it were a value function; it is a value function plus a different constant on every row.

4.7 What the solution looks like

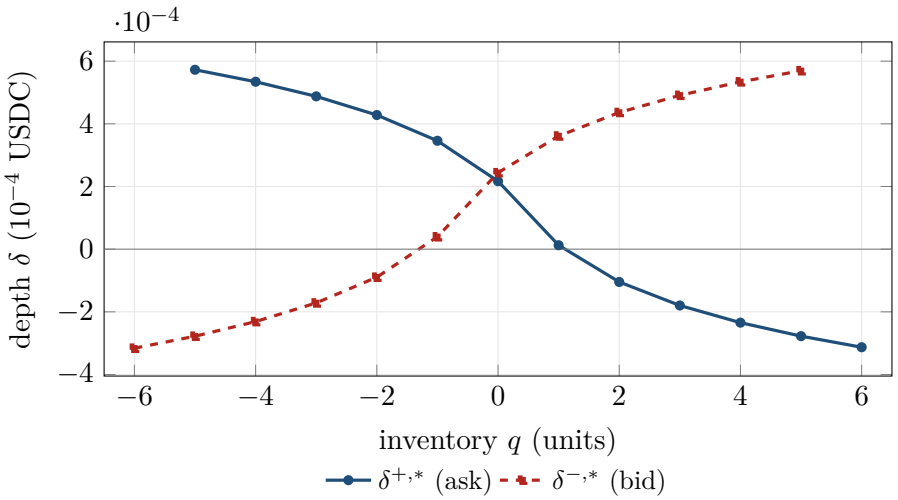

Figure 3: The solved control at the pinned 2026-08-17 CASHCAT calibration, $\tau = 130.8$ s. Long inventory ($q > 0$) tightens the ask and widens the bid, pushing the position back toward zero. The ask is absent at $q = -6$ and the bid at $q = +6$: the inventory boundary switches the adding side off entirely. Depths below zero mean the model would like to quote *through* the mid — see §6.

Figure 3 is the model doing its whole job in one picture. The two curves cross near $q = 0$, where the quotes are symmetric. As inventory grows the ask falls and the bid rises: sell harder, buy more reluctantly.

5 Stage 3: reading $\delta^*(t, q)$

The solver hands back a surface indexed by time and integer inventory. Two questions remain before a number comes out: *which time*, and *which inventory*.

5.1 What one unit of inventory is

Before either question, a quantity that silently scales everything: what does $q = 1$ actually mean in coins?

The book leaves this open — q is simply "units", and $\bar{q}$ bounds it. Here the unit is derived from the account so that a full inventory band is a sensible fraction of capital:

$$u \;=\; \frac{\text{available capital} \times \text{target utilisation} \times \text{target leverage}}{\bar{q} \times S},$$

which at 1000 USDC, 74% utilisation, 2× leverage, $\bar{q} = 6$ and $S = 0.0976$ gives $u \approx 2527$ CASHCAT, so a full ±6 band is about 1480 USDC of notional.

Easy to get backwards

u is the denominator of $q = \text{position}/u$, so changing it re-labels the whole inventory axis — and it also appears in the volatility term $\gamma\sigma^2 u$ of ϕ. Resizing it while holding a position would therefore move the quotes for two unrelated reasons at once. The runtime only ever resizes it **while flat**.

5.2 Which time: the episode clock

The book's control is genuinely time-dependent. It has a deadline T, and the terminal condition $h(T, q) = -\alpha q^2$ shapes everything before it. A perpetual futures contract has no deadline, which leaves a real modelling choice:

- The Python comparison mode `hjb_time_mode = "stationary"` always reads the $t = 0$ slice. The agent then never approaches T, α barely affects the control, and the time axis of (10.26) is discarded.
- The current Rust runtime, and Python's default `"episodic"` mode, run a repeating episode clock of length T and read the real remaining time. The episode restarts at T, or early once the position returns to flat — but only after at least 25% of the horizon has elapsed, so inventory that crosses zero repeatedly cannot restart the clock on every round trip.

Departure from the book

The book liquidates whatever is left at T at market and pays αq^2. Every quote here is **post-only** — an exchange flag meaning "reject this order outright if it would execute on arrival", which guarantees we are always the maker and never the taker. Taking liquidity to flatten would contradict the rest of the design. The terminal condition therefore acts through the *depths* alone, and the clock restarts rather than the position being closed.

5.3 Which inventory: the problem with partial fills

Equation (10.2) is a unit-jump process, so h and δ^* exist only at integer q. Real fills are partial: a resting order for one unit may be filled for 0.4 of it, leaving a fractional position that the integer grid cannot name. Rounding it away discards up to half a unit of live risk – 0.49 units would read as flat.

The fix is to interpolate between the bracketing integers. *Which* quantity gets interpolated matters:

Intuition

Interpolate the **depths**, not h. Since δ reads a *difference* of h across adjacent q, a piecewise-linear h produces piecewise-*constant* depths that jump at the integers – exactly as coarse as the rounding it was meant to replace. Interpolating the depths themselves agrees with the book exactly at every integer and varies smoothly in between.

The book: eq. (10.2): $dQ_t = dN_t^- - dN_t^+$

Inventory moves in unit jumps, so q is an integer and the book's $h(t, q)$ and $\delta^*(t, q)$ are defined **only** at integers. The book never says what to do at $q = 2.4$, because in its world that state cannot occur.

→ **what we wrote**

(continued)

It occurs constantly, because fills are partial. We blend the depths linearly between the two bracketing integers — exact at every integer, an extension of our own in between. The bracketing helper has two guards that look redundant and are not:

scripts/mm_core.py lines 617–628

```
    n = len(axis)
    if n == 1 or value <= axis[0]:
        return 0, 0, 0.0
    if value >= axis[-1]:
        return n - 1, n - 1, 0.0
    hi = int(np.searchsorted(axis, value, side="left"))
    if float(axis[hi]) == float(value):
        return hi, hi, 0.0
    lo = hi - 1
    span = float(axis[hi] - axis[lo])
    weight = 0.0 if span <= 0 else (float(value) - float(axis[lo])) / span
    return lo, hi, weight
```

Easy to get backwards

`side="left"` and the exact-node check protect *opposite* boundaries, and both are needed. Consider a grid $[-3, \dots, 3]$, where δ^+ is infinite at $q = -3$ and δ^- is infinite at $q = +3$.

- At exactly $q = +2$, `side="right"` would return the pair $(2, 3)$ with weight 0. The blend checks finiteness *before* it checks the weight, sees the infinite $q = 3$ node, and switches the **bid off at** $q = 2$ — an inventory the model quotes perfectly happily.
- At exactly $q = -2$, without the exact-node early return you would get the pair $(-3, -2)$ with weight 1, and the infinite $q = -3$ node would switch the **ask off at** $q = -2$.

Returning the node twice consults no neighbour at all, so an integer q reproduces the book exactly and only genuinely fractional inventory blends.

And the blend itself refuses to average away a disabled side:

scripts/mm_core.py lines 645–651

```
    if not np.isfinite(low) or not np.isfinite(high):
        return float("inf")
    if weight <= 0.0:
        return float(low)
    if weight >= 1.0:
        return float(high)
    return float(low) + weight * (float(high) - float(low))
```

Departure from the book

Because *either* infinite endpoint disables the blend, with $\bar{q} = 6$ a bid stops above $q = 5.0$ rather than $q = 5.5$. This is the conservative direction: a bid resting at $q = 5.9$ would permit a jump to 6.9, past the boundary the model is defined on.

5.4 What the time axis actually does here

A natural guess is that the maker should grow *more* aggressive about flattening as $t \to T$, because that is when the terminal penalty αq^2 is about to be charged. It does the reverse, and the reason is worth understanding rather than filing as a bug.

The pinned worked example uses $\phi\kappa T = 10$ and the terminal one $\alpha\kappa = 0.05$: a ratio of 200 to 1. The current Rust profile's 200 makes the dominance stronger. The running penalty is what

remains to be *paid* over the time left, so it is largest at the start of an episode and vanishes at the end. It dominates the surface completely, and the skew therefore collapses as $\tau \to 0$.

Intuition

This is not a quirk of our calibration — it is what the book's own figures show. Figure 10.8 (p. 265) plots exactly this: depth curves for $q = -2 \dots 3$, fanned wide at $t = 0$ and converging into T. Its parameters are $\phi = 0.02$, $\kappa = 100$, $T = 30$, $\alpha = 0.0001$, so $\phi\kappa T = 60$ against $\alpha\kappa = 0.01$ — a ratio of 6000 to 1, even more running-dominated than ours. The book is in the same regime, and our solver reproduces its shape. Treat the table below as a validation, not an anomaly.

τ (s)	$\delta^{+,*}$ at $q = +2$	$\delta^{-,*}$ at $q = -2$
150.0	-1.0464×10^{-4}	-8.9444×10^{-5}
100.0	-1.0464×10^{-4}	-8.9437×10^{-5}
50.0	-1.0443×10^{-4}	-8.9156×10^{-5}
10.0	-5.6830×10^{-5}	-3.9855×10^{-5}
1.0	$+8.6643 \times 10^{-5}$	$+9.6637 \times 10^{-5}$
0.0	$+1.0852 \times 10^{-4}$	$+1.1740 \times 10^{-4}$

Table 6: Depth against time-to-go. The unwinding pressure is strongest at the *start* of an episode and relaxes into the terminal, matching the shape of the book's Figure 10.8 — which is in the same running-penalty-dominated regime.

The last row of Table 6 is a useful check on the whole machine. At $\tau = 0$ we have $h(T, q) = -\alpha q^2$ exactly, so the ask half of (10.27) collapses to a closed form:

$$\delta^{+,*}(T, q) = \frac{1}{\kappa^+} + \bar{\varepsilon}^+ + \alpha(1 - 2q).$$

At $q = 2$ with the worked-example numbers that is $9.8783627 \times 10^{-5} + 2.5034909 \times 10^{-5} - 3 \times 5.1000538 \times 10^{-6} = 1.0851837 \times 10^{-4}$, against the solver's 1.0851837×10^{-4} – agreement to seven significant figures.

Departure from the book

The Python field `hjb_alpha_kappa` (Rust: `alpha_kappa`) was set to 0.05 when only the $t = 0$ slice was read, so the value carries no tuning evidence. It affects episodic quotes and remains unswept; treat it as untuned.

6 Stages 4 and 5: from a depth to a resting order

The model's δ is not yet a price. Four things happen to it.

6.1 The arithmetic

The book: nothing at all

This is the one stage with no equation behind it. In the book, δ^* *is* the answer: the agent posts at $S \pm \delta^*$ and the model is done. There are no fees, no tick sizes, no minimum spreads and no exchange that rejects orders.

→ what we wrote

(continued)

Everything below is ours, and every line of it can override the model:

`scripts/mm_core.py` lines 315–329

```
fee_cushion = float(config.maker_fee_rate) * mid
extra_cushion = float(config.extra_cushion_bps) / 10_000.0 * mid
delta_pre_clamp = model * float(config.spread_multiplier) + fee_cushion + extra_cushion
floor = float(config.min_half_spread_bps) / 10_000.0 * mid
cap = float(config.max_half_spread_bps) / 10_000.0 * mid
delta_total = min(max(delta_pre_clamp, floor), cap)

clamped = None
if delta_pre_clamp < floor:
    clamped = "floor"
elif delta_pre_clamp > cap:
    clamped = "cap"

p95 = finite_float_or_none(depth_p95)
```

Step	Expression	Pinned example
model term	$\delta_{\text{model}} \times$ `spread_multiplier`	1.0
fee cushion	`maker_fee_rate` $\times$ mid	1.5 bps†
extra cushion	`extra_cushion_bps`$/10^4 \times$ mid	0
floor	`min_half_spread_bps`$/10^4 \times$ mid	1.5 bps
cap	`max_half_spread_bps`$/10^4 \times$ mid	80 bps

† The live backend loads the account's actual maker fee. The worked example and replay default pin the 1.5 bps rate measured for the CASHCAT account on 2026-08-23.

Four design choices are encoded here:

- **The fee cushion is one maker fee per side.** A round trip pays two fees and collects the cushion twice.
- **The multiplier scales only the model term**, so widening quotes defensively never inflates the fee compensation.
- **Defensive widening should use `extra_cushion_bps`, not the floor.** The extra cushion is *additive*: it shifts both sides equally and leaves the inventory skew of Figure 3 intact. Raising `min_half_spread_bps` is a *clamp*, and a clamp flattens the skew for every q beneath it.
- **A disabled side returns nothing at all.** When δ is infinite the function returns `None` rather than clamping, because clamping ∞ into $[\text{floor}, \text{cap}]$ would silently turn "do not quote" into "quote at the floor".
- **The pinned example's floor equals one maker fee.** `min_half_spread_bps` and the replay's `maker_fee_rate` are both 1.5 bps, so the floor and fee cushion are equal in this calculation. This is not a live invariant: an account-specific fee change updates the cushion but not the configured floor.

How often the model is actually overruled

Work out where that floor binds in the pinned example and the answer is larger than it looks. Because floor = fee cushion there, $\delta_{\text{total}} = \max(\delta_{\text{model}} + \text{fee},\ \text{fee})$, so **the clamp binds precisely when the model asks for a negative depth** — nothing more, nothing

(continued)

less.

Figure 3's ask curve crosses zero at $q = 1.107$. So at this τ, for every inventory above about 1.1 units — four-fifths of the positive band — the ask is pinned at 1.5 bps and carries **no inventory information at all**, and the bid mirrors it below -1.1. The skew that §2 spends fifteen pages deriving reaches the market, on the reducing side, only over roughly $|q| < 1.1$; outside that band one side is a constant and the model's entire remaining influence is in how far it widens the other.

Figure 3 therefore draws the *model*, not the *behaviour*. For the behaviour, clip it at zero and add 1.5 bps. Note the interaction with §5.4 as well: the clamp releases as $\tau \to 0$, so the model is most overruled at exactly the moments it is most aggressive.

Intuition

Figure 3 shows the model asking for *negative* ask depths once $q \geq 2$. A negative depth means "cross the mid to get out". That is a coherent instruction – the model wants the inventory gone – but this strategy is post-only, so it cannot be obeyed. The floor is what stops it, and the `clamped` flag records that the model was overruled.

6.2 Rounding, safely

`scripts/mm_core.py` lines 78–95

```
def round_price_for_side(
    *,
    side: str,
    price: float,
    price_tick_size: float | None,
    rounding_policy: str = "maker_safe",
) -> float:
    """Round price without weakening the intended post-only safety property."""
    if price_tick_size is None or float(price_tick_size) <= 0:
        return float(price)
    is_buy = side_is_buy(side)
    if rounding_policy == "maker_safe":
        rounding = ROUND_FLOOR if is_buy else ROUND_CEILING
    elif rounding_policy == "crossing_probe":
        rounding = ROUND_CEILING if is_buy else ROUND_FLOOR
    else:
        raise ValueError(f"unsupported rounding_policy: {rounding_policy}")
    return _round_to_step(float(price), float(price_tick_size), rounding=rounding)
```

Bids round *down*, asks round *up* – always away from the touch, never toward it, and in `Decimal` rather than binary floating point. A bid rounded one unit in the last place the wrong way can cross the ask and turn a post-only order into an exchange rejection.

6.3 The last check before posting

`scripts/mm_core.py` lines 358–369

```
    price_f = finite_float_or_none(price)
    bid_f = finite_float_or_none(best_bid)
    ask_f = finite_float_or_none(best_ask)
    if price_f is None or price_f <= 0:
        return False, "invalid_rate"
    if bid_f is None or ask_f is None or bid_f <= 0 or ask_f <= 0 or bid_f >= ask_f:
        return False, "crossed_or_invalid_book"
    if side == "bid" and price_f >= ask_f:
        return False, "bid_crosses_ask"
    if side == "ask" and price_f <= bid_f:
        return False, "ask_crosses_bid"
```

```
    return True, "ok"
```

The comparisons are non-strict: a bid resting exactly *at* the best ask would cross, so equality is rejected too. A crossed or otherwise invalid book fails closed.

Easy to get backwards

The superscripts name the incoming market order, while the quote names the resting side: a buy MO lifts our ask, so the ask uses δ^+ and price $S + \delta^+$; a sell MO hits our bid, so the bid uses δ^- and price $S - \delta^-$. The depth and price sign must flip together. Mixing them puts a quote on the wrong side of the book, which the maker-safety check rejects.

7 A fully worked example

Everything below is produced by the executable Python replay/reference pricing path, and every tagged number is re-checked by `scripts/verify_spread_doc.py`. Rust parity is tested separately.

7.1 Inputs

One recorded CASHCAT estimator cycle, stamped 2026-08-17T16:20:41Z over a 120-minute window.

κ^+	10,123.135	λ^+	0.113130
κ^-	9,484.501	λ^-	0.122767
$\bar{\varepsilon}^+$	2.5035×10^{-5}	σ^2	3.3370×10^{-9}
$\bar{\varepsilon}^-$	2.7263×10^{-5}	mid S	0.105685
T	150 s	unit size u	2527.32 CASHCAT
$\bar{q}$	6	tick	10^{-5}
$(\phi\kappa T)_{\text{target}}$	10	$(\alpha\kappa)_{\text{target}}$	0.05

Inventory is **6065.6 CASHCAT**, deliberately not a whole number of units – this is what a partial fill leaves behind, and it is what the interpolation exists for. Time-to-go is $\tau = 130.79$ s.

7.2 Step 1 — the risk parameters

Quantity	Formula	Value
$\bar{\kappa}$	$\frac{1}{2}(\kappa^+ + \kappa^-)$	9803.8181
$\phi_{\kappa\text{-rel}}$	$10/(\bar{\kappa}T)$	6.8001×10^{-6}
volatility term	$\gamma\sigma^2 u$	4.2168×10^{-7}
ϕ	sum of the two	7.2218×10^{-6}
α	$0.05/\bar{\kappa}$	5.1001×10^{-6}
`phi_source`	—	`kappa_relative+sigma2`
`n_steps`	$\lceil T/0.25 \rceil$	600
dt	$T/$`n_steps`	0.25 s

The volatility channel contributes about 6% of ϕ here, lifting the dimensionless product from the configured 10 to $\phi\bar{\kappa}T = 10.6201$ – comfortably below the cap of 50.

7.3 Step 2 — inventory

q_{exact}	6065.6/2527.32	2.4
q (integer, for routing)	round(q_{exact})	2
q_{residual}	$q_{\text{exact}} - q$	0.4

That residual of 0.4 units is roughly 1010 CASHCAT of live exposure that the integer grid cannot represent. It is logged on every quote, and a run whose mean $|q_{\text{residual}}|$ exceeds 0.35 fails the dry-run quality gate.

7.4 Step 3 — where the depth comes from

Before blending, look at how the model actually builds the ask at the integer node $q = 2$. Equation (10.27) has three terms and they are nowhere near the same size:

Term	What it is	USDC	bps of mid
$1/\kappa^+$	rent	$+9.8784 \times 10^{-5}$	$+9.35$
$\bar{\varepsilon}^+$	adverse-selection cover	$+2.5035 \times 10^{-5}$	$+2.37$
$-\big(h(\tau,1) - h(\tau,2)\big)$	inventory pressure	-2.2846×10^{-4}	-21.62
$\delta^{+,*}(\tau,2)$		-1.0464×10^{-4}	-9.90

The inventory term is nearly twice the other two combined, and it points the other way. **That is the answer to why the ask gets clamped two steps from now**: at $q = 2$ the model does not want a spread, it wants *out*, and it is willing to pay 9.9 bps through the mid to get there. The first two terms — the entire content of §2 up to (10.27) — are worth 11.7 bps between them and are simply overwhelmed. Everything that makes this a *strategy* rather than a fixed spread is in the third row.

7.5 Step 3b — blending to the real inventory

The fractional inventory sits between the $q = 2$ and $q = 3$ nodes, so each depth is a blend with weight 0.4:

	at $q = 2$	at $q = 3$	blended at $q = 2.4$
$\delta^{+,*}$ (ask)	-1.0464×10^{-4}	-1.7974×10^{-4}	-1.3468×10^{-4}
$\delta^{-,*}$ (bid)	4.3626×10^{-4}	4.9105×10^{-4}	4.5817×10^{-4}

Check the ask by hand: $-1.0464 \times 10^{-4} + 0.4 \times (-1.7974 + 1.0464) \times 10^{-4} = -1.3468 \times 10^{-4}$. Note it is **negative**: holding 2.4 units long, the model wants to sell badly enough to cross the mid.

7.6 Step 4 — assembling both prices

	bid	ask
model depth δ_{model}	4.5817×10^{-4}	-1.3468×10^{-4}
+ fee cushion	1.5853×10^{-5}	1.5853×10^{-5}
= pre-clamp	4.7403×10^{-4}	-1.1883×10^{-4}
floor (1.5 bps)	1.5853×10^{-5}	
cap (80 bps)	8.4548×10^{-4}	
clamped?	no	**yes, at the floor**
final δ_{total}	4.7403×10^{-4}	1.5853×10^{-5}
in bps of mid	44.85	1.50
raw price ($S \mp \delta$)	0.10521097	0.10570085
rounded to tick	0.10521 (down)	0.10571 (up)

7.7 Reading the result

The final quote is wildly asymmetric: the ask sits 1.50 bps from the mid and the bid 44.85 bps away. Those sum to 46.35 bps, while the quoted spread measured between the two *rounded* prices is 47.31 bps — the extra 0.96 bps is tick rounding pushing both prices away from the touch, which at a 10^{-5} tick on a 0.1057 mid is worth about half a basis point per side.

None of that is a malfunction; the asymmetry is the entire point. Holding 2.4 units long, the strategy is almost giving the inventory away on the ask while making itself very hard to hit on the bid.

Intuition

Put the two final depths back through (D1) and ask what they are worth. The ask, pinned at the floor, runs at $\lambda^{+}e^{-\kappa^{+}\delta} = 0.0964\ \mathrm{s}^{-1}$ — a fill every 10.4 seconds. The bid, 44.85 bps out, runs at $0.00137\ \mathrm{s}^{-1}$: a fill every 12 minutes, against an episode horizon of 150 seconds. The bid is not really a quote. It is a decision not to buy, dressed as one. That is correct behaviour at 2.4 units long, and it is the clearest statement of what the inventory term does: it does not merely lean the book, it switches a side off in practice long before the hard boundary at $\bar{q}$ ever does.

7.8 The same machine at four inventories

The example above shows the model being overruled. It is worth seeing it work unobstructed too, so here is the identical pipeline at four inventories:

q	bid (bps)	ask (bps)	clamped?	
0.0	24.58	22.03	no	symmetric; the model as the book intends it
0.4	29.02	14.29	no	mild skew, both sides live
2.4	44.85	1.50	ask	the worked example
5.4	—	1.50	ask	bid switched off by the boundary blend

Three things here that no prose in this document can show as well: the skew is real and gradual near flat; the boundary blend genuinely disables the bid at 5.4, exactly as §5 warned; and **the ask is identical at $q = 2.4$ and $q = 5.4$**, because the clamp has flattened it. Between those two inventories the model has a great deal more to say and no way to say it.

7.9 Diagnostics

Two diagnostics fire on the $q = 2.4$ quote and both are worth attention:

- **The ask was clamped at the floor.** The model asked for a negative depth and got 1.5 bps instead. The final quote is therefore *not* following the model at this inventory — it is following the constraint. If this fires often, the model is telling you the inventory band is too wide for the liquidity available.
- **The bid is outside the calibrated range.** At 4.74×10^{-4} it sits beyond the 95th percentile of observed market-order walk depths (3.42×10^{-4}). Equation (D1) is being extrapolated past the data that fitted it, so the true fill probability there is anybody's guess.

8 Where this departs from the book

A faithful implementation is the goal, so the gaps are listed rather than glossed.

1. **No forced liquidation at** T, as described in §5.

2. **Fractional inventory is interpolated** over a domain the book does not define. Exact at every integer q; an extension in between.

3. **The fee cushion and the bps clamps** are outside the model entirely. The clamps in particular *override* the model whenever it asks for a depth below the floor, which the worked example shows is not rare.

4. **Leverage, margin, and perpetual funding** do not appear in the control. Runtime risk checks enforce the first two and accounting charges the third, but none changes the HJB depths.

5. **Parameters are re-estimated every 30 seconds** and the problem re-solved. The agent therefore follows a *sequence* of solutions to slightly different problems, not one solution to a fixed problem.

6. **Quoting is event-driven and discrete, not continuous.** Venue acknowledgement/-cancel latency, minimum order lifetime, and requote hysteresis determine how long a target remains exposed. The Rust quote computation is much faster than those effects, but it cannot remove them.

7. **Queueing, fill participation, risk gates, and the toxic-flow guard** are empirical/runtime mechanisms outside the book's control problem.

Intuition

Adverse selection roughly *doubles* between a 200 ms and a 5 s markout on this instrument. The model prices $\bar{\varepsilon}$ at the 200 ms horizon because that is what (10.22) means, while a resting quote remains exposed to subsequent flow until it is filled or cancelled. That makes latency and hold policy scientifically relevant, but not automatically profitable: the 161.95-hour replay reversed the earlier latency ordering, and the 185-hour width–cadence matrix was non-monotone. The defensible conclusions come from measured markouts, independent frozen windows, and explicit scenario assumptions, not from a generic "faster is always better" rule.

Not a departure, despite appearances: the **volatility term in** ϕ. §10.4.1 treats ϕ as a constant, but §10.3 derives $\phi = \frac{1}{2}\sigma^2\gamma$ (§2.5.1), so a ϕ that scales with variance is the book's own correspondence rather than an outside graft. The code's constant differs ($\gamma\sigma^2 u$ rather than $\frac{1}{2}\sigma^2\gamma$, with the unit size folded in to match h's per-unit normalisation), so $\gamma_{\text{inventory-risk}}$ is a knob of its own, not literally the book's risk aversion.

A File map

File	Role
`scripts/hyperliquid_data_collector.py`	L2 book and trade capture to Parquet
`scripts/estimator_common.py`	window loading, κ, σ^2
`scripts/get_kappa.py`	$\kappa^{\pm}$, $\lambda^{\pm}$, σ^2 driver
`scripts/get_epsilon.py`	$\bar{\varepsilon}^{\pm}$ driver
`scripts/hjb.py`, `scripts/mm_core.py`	Python solver, risk scaling, depth lookup, and quote assembly
`scripts/replay_market_maker.py`	Python event replay of the whole pipeline
`rust_live/crates/cj-data/`	Rust Parquet calibration and replay
`rust_live/crates/cj-core/`	Rust HJB, instrument math, and quote policy
`rust_live/src/main.rs`	runtime orchestration, dry-run grid, and live command
`scripts/verify_spread_doc.py`	checks this document against the code

B Rebuilding this document

```
cd docs
latexmk -pdf spread_calculation.tex
python ../scripts/verify_spread_doc.py
```

The verifier re-reads every source range quoted above and re-runs the pricing code to regenerate every number in §7. If a refactor moves a function or a default changes, it fails.

C Further reading

Cartea, Á., Jaimungal, S. and Penalva, J. (2015). *Algorithmic and High-Frequency Trading*. Cambridge University Press. Chapter 10 is market making; §10.2 is the base model, §10.3 the utility-maximising equivalence, §10.4 adverse selection, and §10.4.1 the model implemented here.

Avellaneda, M. and Stoikov, S. (2008), "High-frequency trading in a limit order book", *Quantitative Finance* 8(3). The ancestor of this model: the same $\lambda e^{-\kappa\delta}$ fill law and inventory skew, without the adverse-selection jumps of (10.22). Reading it first makes clear which parts of Chapter 10 are the jumps' doing.

Guéant, O., Lehalle, C.-A. and Fernandez-Tapia, J. (2013), "Dealing with the inventory risk". Takes the same problem to closed form under a symmetry assumption, and is the standard reference for the inventory-bound treatment.

`docs/market_making_introduction.ipynb` is a French-language background notebook with an interactive model dashboard. It predates the current Rust runtime and is not the authority for configuration or profitability thresholds, but remains useful for seeing how κ, λ, $\bar{\varepsilon}$, ϕ and α move the idealised depths.

On measuring toxicity more generally, VPIN (Easley, López de Prado and O'Hara, 2012, "Flow toxicity and liquidity in a high-frequency world") is the standard alternative to the $\kappa\bar{\varepsilon}$ number used here: it estimates the probability that flow is informed from volume imbalance rather than from post-trade markouts, and does not need a fitted κ.